

Ejercicio 1: Análisis de Arquitectura MCP

Escenario: una empresa de comercio electrónico desea integrar un asistente de IA que pueda consultar inventario, procesar pedidos y responder preguntas frecuentes de clientes.

Diagrama de arquitectura

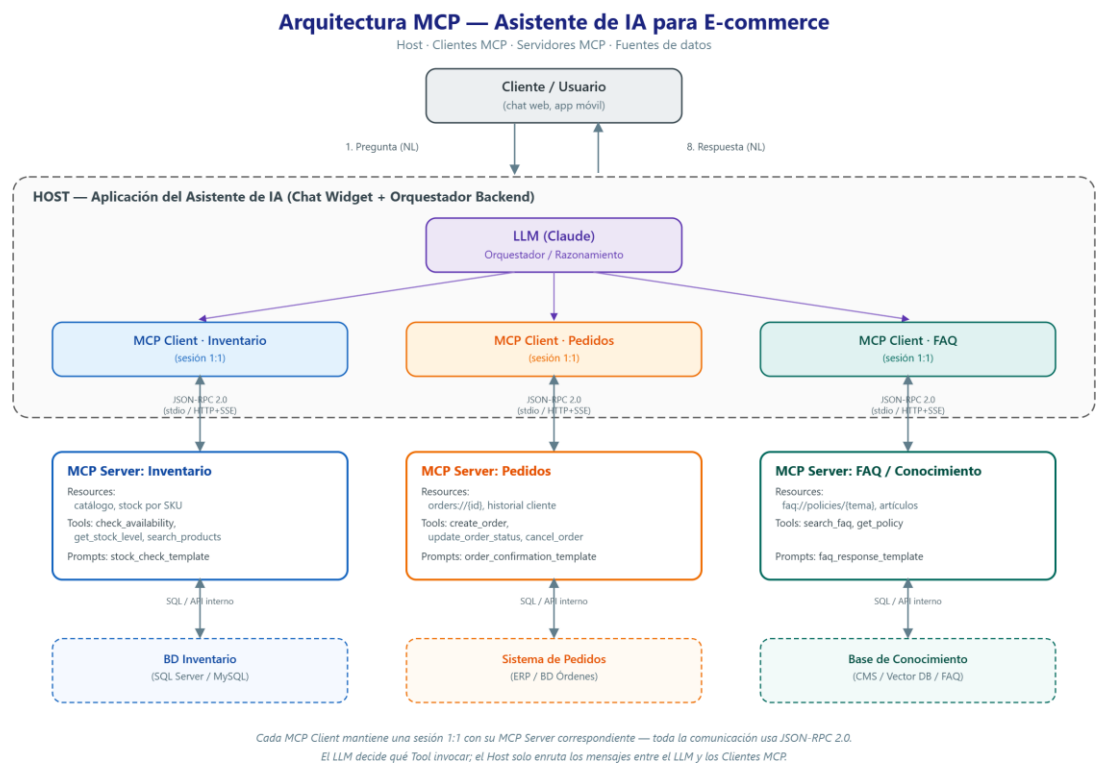


Figura 1. Arquitectura MCP para el asistente de e-commerce: Host, Clientes MCP, Servidores MCP y fuentes de datos.

Servidores MCP identificados

Se identificaron tres servidores MCP distintos, cada uno especializado en un dominio del negocio:

Servidor	Propósito	Resources	Tools	Prompts
mcp-inventory-server	Consultar catálogo, stock y disponibilidad por talla/color	inventory://products inventory://product/{sku} inventory://stock/{sku}	check_availability search_products reserve_stock	product_comparison low_stock_alert

mcp-orders-server	Crear, consultar y gestionar pedidos	orders://customer/{id} orders://order/{id} orders://status/{id}	create_order cancel_order process_payment	order_confirmation order_status_summary
mcp-support-server	Responder FAQ, políticas y escalar casos	support://faq support://policies/returns support://policies/shipping	search_faq create_support_ticket	customer_response_template escalation_summary

Flujo de comunicación (JSON-RPC)

Cuando el cliente pregunta «¿Tienen disponible el producto X en talla M?», el flujo es el siguiente:

1. El usuario envía la pregunta en lenguaje natural al Host, que la pasa al LLM.
2. El LLM decide invocar la tool `check_availability` del servidor `mcp-inventory-server` (la sesión con ese servidor ya fue inicializada previamente con `initialize / notifications/initialized`). El Cliente MCP correspondiente envía una solicitud `tools/call`:

```
{
  "jsonrpc": "2.0",
  "id": 17,
  "method": "tools/call",
  "params": {
    "name": "check_availability",
    "arguments": { "product_id": "X", "size": "M" }
  }
}
```

3. El servidor de Inventario consulta la base de datos y responde con un resultado `tools/call`:

```
{
  "jsonrpc": "2.0",
  "id": 17,
  "result": {
    "content": [{ "type": "text", "text": "Sí hay 8 unidades del producto X en talla M." }],
    "isError": false
  }
}
```

4. El LLM recibe el resultado y redacta la respuesta final en lenguaje natural para el usuario a través del Host: «Sí, tenemos el producto X disponible en talla M».

Justificación: ¿por qué MCP en vez de integraciones API por API?

Para este escenario, MCP resulta más conveniente que integrar cada servicio (inventario, pedidos, soporte) con su propia API REST individual, por cuatro razones concretas. Primero, el LLM descubre automáticamente las tools disponibles de cada servidor mediante la negociación de capacidades en la fase de inicialización, sin que el equipo de desarrollo tenga que mantener documentación de API separada para cada integración. Segundo, agregar un cuarto servicio (por ejemplo, un sistema de devoluciones) solo implica levantar un nuevo servidor MCP y conectarlo, sin modificar el código del Host ni del LLM — con REST habría que escribir un nuevo cliente HTTP para cada servicio nuevo. Tercero, el modelo de permisos de MCP exige confirmación antes de ejecutar tools con efectos secundarios (como `create_order`), algo que una integración REST directa no garantiza por sí sola. Cuarto, al usar un único protocolo (JSON-RPC 2.0) para los tres dominios, el equipo evita mantener tres esquemas de autenticación y manejo de errores distintos.

Ejercicio 2: Implementación de un Servidor MCP Básico

Código fuente completo del servidor, con historial de commits y README de instalación/ejecución: <https://github.com/dorlingm07/Extraclase-1-MCP>

Capacidades implementadas

Resource `tasks://pending` — retorna en JSON las tareas pendientes almacenadas en `tasks.json`.

Tool `add_task(name, description, priority)` — agrega una nueva tarea (prioridad alta, media o baja).

Tool `complete_task(task_id)` — marca una tarea como completada.

Prompt `daily_summary` — genera un resumen del estado actual de las tareas (total, pendientes por prioridad, completadas).

Implementado en Python con el SDK oficial (`mcp==2.0.0`), usando la clase `MCPServer` (`mcp.server.mcpserver`) sobre transporte `stdio`.

Evidencia de instalación y ejecución

Instalación del entorno virtual y dependencias:

```

PS C:\Users\dorli\OneDrive\Desktop\ExtraClase1P4> cd mcp-todo-server
>> python -m venv venv
>> venv\Scripts\activate
>> pip install -r requirements.txt
>> python test_client.py
Requirement already satisfied: mcp==2.0.0 in .\venv\Lib\site-packages (from mcp[cli]==2.0.0->r requirements.txt (line 1)) (2.0.0)
Requirement already satisfied: anyio>=4.9 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (4.14.2)
Requirement already satisfied: httpx2>=2.5.0 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (2.12.0)
Requirement already satisfied: jsonschema>=4.20.0 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (4.26.0)
Requirement already satisfied: mcp-types==2.0.0 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (2.0.0)
Requirement already satisfied: opentelemetry-api>=1.28.0 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (1.44.0)
Requirement already satisfied: pydantic>=2.12.0 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (2.13.4)
Requirement already satisfied: pyjwt>=2.10.1 in .\venv\Lib\site-packages (from pyjwt[crypto]>=2.10.1->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (2.13.0)
Requirement already satisfied: python-multipart>=0.0.9 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (0.0.32)
Requirement already satisfied: pywin32>=311 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (312)
Requirement already satisfied: sse-starlette>=3.0.0 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (3.4.8)
Requirement already satisfied: starlette>=0.27 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (1.6.0)
Requirement already satisfied: typing-extensions>=4.13.0 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (4.16.0)
Requirement already satisfied: typing-inspection>=0.4.1 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (0.4.4)
Requirement already satisfied: uvicorn>=0.31.1 in .\venv\Lib\site-packages (from mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (0.52.4)
Requirement already satisfied: python-dotenv>=1.0.0 in .\venv\Lib\site-packages (from mcp[cli]==2.0.0->r requirements.txt (line 1)) (1.2.3)
Requirement already satisfied: typer>=0.16.0 in .\venv\Lib\site-packages (from mcp[cli]==2.0.0->r requirements.txt (line 1)) (0.27.1)
Requirement already satisfied: idna>=2.8 in .\venv\Lib\site-packages (from anyio>=4.9->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (3.19)
Requirement already satisfied: httpcore2==2.12.0 in .\venv\Lib\site-packages (from httpx2>=2.5.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (2.12.0)
Requirement already satisfied: truststore>=0.10 in .\venv\Lib\site-packages (from httpx2>=2.5.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (0.10.4)
Requirement already satisfied: h11>=0.16 in .\venv\Lib\site-packages (from httpcore2==2.12.0->httpx2>=2.5.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (0.16.0)
Requirement already satisfied: attrs>=22.2.0 in .\venv\Lib\site-packages (from jsonschema>=4.20.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (26.1.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in .\venv\Lib\site-packages (from jsonschema>=4.20.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in .\venv\Lib\site-packages (from jsonschema>=4.20.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (0.37.0)
Requirement already satisfied: rpds-py>=0.25.0 in .\venv\Lib\site-packages (from jsonschema>=4.20.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (2026.6.3)
Requirement already satisfied: annotated-types>=0.6.0 in .\venv\Lib\site-packages (from pydantic>=2.12.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (0.8.0)
Requirement already satisfied: pydantic-core==2.46.4 in .\venv\Lib\site-packages (from pydantic>=2.12.0->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (2.46.4)
Requirement already satisfied: cryptography>=3.4.0 in .\venv\Lib\site-packages (from pyjwt[crypto]>=2.10.1->mcp==2.0.0->mcp[cli]==2.0.0->r requirements.txt (line 1)) (50.0.0)

```

Ejecución de test_client.py mostrando las 4 interacciones (Resource, ambas Tools y el Prompt) por protocolo MCP real:

```
Requirement already satisfied: cffi>=2.0.0 in .\venv\Lib\site-packages (from cryptography>=3.4.0->pyjwt[crypto]>=2.10.1->mcp==2.0.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (2.1.1)
Requirement already satisfied: pycparser in .\venv\Lib\site-packages (from cffi>=2.0.0->cryptography>=3.4.0->pyjwt[crypto]>=2.10.1->mcp==2.0.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (3.0)
Requirement already satisfied: shellingham>=1.3.0 in .\venv\Lib\site-packages (from typer>=0.16.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in .\venv\Lib\site-packages (from typer>=0.16.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (15.0.0)
Requirement already satisfied: annotated-doc>=0.0.2 in .\venv\Lib\site-packages (from typer>=0.16.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (0.0.5)
Requirement already satisfied: colorama in .\venv\Lib\site-packages (from typer>=0.16.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (0.4.6)
Requirement already satisfied: markdown-it-py>=2.2.0 in .\venv\Lib\site-packages (from rich>=13.8.0->typer>=0.16.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in .\venv\Lib\site-packages (from rich>=13.8.0->typer>=0.16.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (2.21.0)
Requirement already satisfied: mdurl<=0.1 in .\venv\Lib\site-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer>=0.16.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (0.1.2)
Requirement already satisfied: click>=7.0 in .\venv\Lib\site-packages (from uvicorn>=0.31.1->mcp==2.0.0->mcp[cli]==2.0.0->-r requirements.txt (line 1)) (8.4.2)
Conectado a 'todo-server' (protocolo 2025-11-25)

=== Interacción 1: Resource tasks://pending ===
[
  {
    "id": 1,
    "name": "Preparar entorno de desarrollo",
    "description": "Instalar Python y las dependencias del servidor MCP",
    "priority": "alta",
    "completed": false
  }
]

=== Interacción 2: Tool add_task ===
Tarea #3 'Probar servidor MCP' agregada con prioridad media.

=== Interacción 3: Tool complete_task ===
Tarea #1 'Preparar entorno de desarrollo' marcada como completada.

=== Interacción 4: Prompt daily_summary ===
Genera un resumen diario del estado de las tareas con estos datos:
- Total de tareas: 3
- Pendientes: 1 (alta: 0, media: 1, baja: 0)
- Completadas: 2

Detalle de pendientes:
#3 [media] Probar servidor MCP: Verificar que las tools respondan correctamente
(PowerShell) PS C:\Users\dorli\OneDrive\Desktop\ExtraClaseIP4\mcp-todo-server>
```

tasks.json después de la ejecución, mostrando que los cambios persisten en disco:

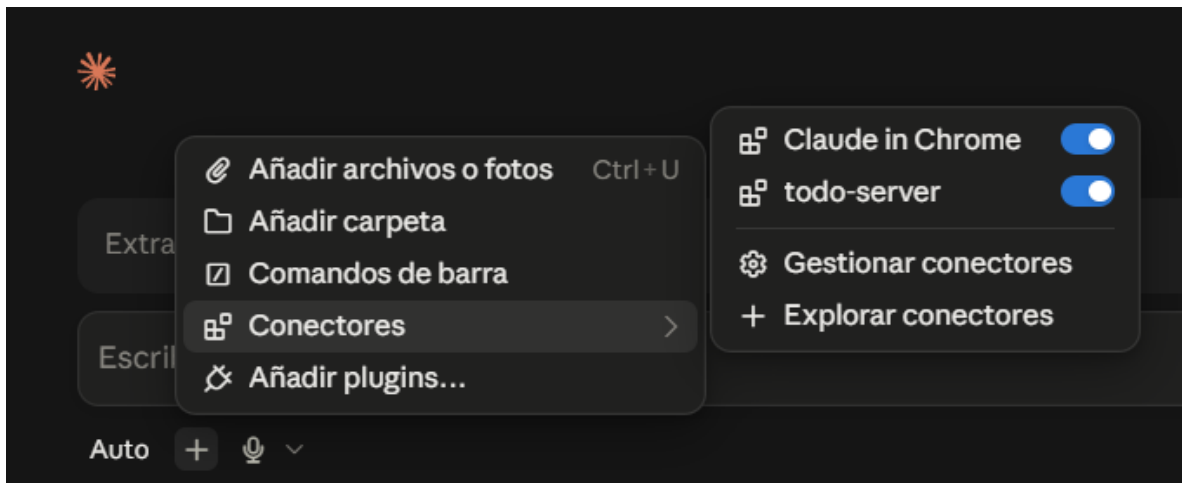
```
mcp-todo-server > {} tasks.json
1  {
2    "tasks": [
3      {
4        "id": 1,
5        "name": "Preparar entorno de desarrollo",
6        "description": "Instalar Python y las dependencias del servidor MCP",
7        "priority": "alta",
8        "completed": true
9      },
10     {
11       "id": 2,
12       "name": "Documentar el servidor",
13       "description": "Escribir el README con instrucciones de instalación y uso",
14       "priority": "media",
15       "completed": true
16     },
17     {
18       "id": 3,
19       "name": "Probar servidor MCP",
20       "description": "Verificar que las tools respondan correctamente",
21       "priority": "media",
22       "completed": false
23     }
24   ],
25   "next_id": 4
26 }
```

Ejercicio 3: Integración con un Cliente MCP

Configuración del cliente

Se registró el servidor todo-server en claude_desktop_config.json:

```
"mcpServers": {  
  "todo-server": {  
    "command": "...\\mcp-todo-server\\venv\\Scripts\\python.exe",  
    "args": ["...\\mcp-todo-server\\server.py"]  
  }  
}
```



Operaciones con Tools (add_task, complete_task)

Se agregaron dos tareas nuevas y se completó una, invocadas directamente desde el chat de Claude Desktop conectado al servidor:

```
Found tools >  
Perfecto, hago las dos tareas nuevas y luego completo una — así queda registrado en esta misma conversación.  
Usado 2 herramientas >  
Dos tareas agregadas (#4 y #5). Ahora completo la #3, que ya probamos que funcionaba en el Ejercicio 2.  
Usó todo-server: Complete task >  
Las 3 tools ya quedaron documentadas en este chat (dos add_task + un complete_task) — captura esta parte de la conversación para el ejercicio. Ahora reviso si el resource y el prompt también están disponibles como acciones directas aquí.
```

Operaciones con Resource y Prompt (cliente alterno)

Claude Desktop no expone una interfaz para Resources ni Prompts, solo para Tools. Por eso estas dos operaciones se probaron con un cliente MCP alterno (script Python, test_client.py), tal como permite el enunciado («u otro cliente MCP compatible»):


```
(venv) PS C:\Users\dorli\OneDrive\Desktop\ExtraClase1P4\mcp-todo-server> cd mcp-todo-server
>> venv\Scripts\activate
>> python test_client.py
```

Conectado a 'todo-server' (protocolo 2025-11-25)

=== Interacción 1: Resource tasks://pending ===

```
[
  {
    "id": 4,
    "name": "Configurar Claude Desktop",
    "description": "Registrar el servidor todo-server en claude_desktop_config.json",
    "priority": "alta",
    "completed": false
  },
  {
    "id": 5,
    "name": "Tomar capturas del cliente MCP",
    "description": "Documentar las 4 interacciones para el Ejercicio 3",
    "priority": "media",
    "completed": false
  },
  {
    "id": 6,
    "name": "Probar servidor MCP",
    "description": "Verificar que las tools respondan correctamente",
    "priority": "media",
    "completed": false
  }
]
```

=== Interacción 2: Tool add_task ===

Tarea #7 'Probar servidor MCP' agregada con prioridad media.

=== Interacción 3: Tool complete_task ===

La tarea #1 ya estaba completada.

=== Interacción 4: Prompt daily_summary ===

Genera un resumen diario del estado de las tareas con estos datos:

- Total de tareas: 7
- Pendientes: 4 (alta: 1, media: 3, baja: 0)
- Completadas: 3

Prueba del Resource tasks://pending y el Prompt daily_summary mediante un cliente MCP alternativo (script Python), ya que Claude Desktop no expone una interfaz para estas dos primitivas — solo para Tools.

Análisis comparativo: MCP vs. API REST

Al implementar y probar mi propio servidor MCP, pude comparar en la práctica esta tecnología con el enfoque tradicional de consumir una API REST, y encontré ventajas y desventajas concretas.

La primera ventaja fue el descubrimiento automático de capacidades. Apenas conecté mi servidor "todo-server" a Claude Desktop, las herramientas `add_task` y `complete_task` quedaron disponibles de inmediato, sin que yo escribiera documentación adicional. Con una API REST tradicional habría tenido que documentar cada ruta, por ejemplo con Swagger, para que cualquier consumidor supiera usarla. Otra ventaja fue la interoperabilidad: usé el mismo `server.py`, sin modificar una línea, tanto desde Claude Desktop como desde un script cliente en Python, y ambos leyeron y escribieron el mismo estado sin problema. Con REST habría necesitado un cliente distinto para cada consumidor. También valoro que MCP estandariza las interacciones en tres primitivas claras, Resources, Tools y Prompts, en vez de que cada desarrollador invente su propio esquema de rutas.

También encontré desventajas reales. La más notoria fue que el soporte de MCP varía según el cliente: descubrí que Claude Desktop solo expone las Tools en su interfaz, pero no muestra Resources ni Prompts, algo que sí pude probar con un cliente de línea de comandos hecho por mí. Con REST, cualquier cliente HTTP accede igual a todos los endpoints. Además, el ecosistema de MCP aún es inmaduro: el SDK de Python cambió su API principal, de FastMCP a MCPServer, entre versiones recientes, rompiendo varios tutoriales que consulté. Por último, la configuración inicial fue más pesada de lo esperado: instalar el SDK, crear un entorno virtual y editar el archivo de configuración del cliente implicó más pasos que una simple petición HTTP.

En conclusión, MCP conviene cuando un modelo de lenguaje necesita descubrir y usar herramientas externas de forma dinámica y estandarizada, pero REST sigue siendo más simple, predecible y maduro para integraciones punto a punto tradicionales.